

Laboratorio 2 de Simulación Computacional

Números (pseudo)aleatorios y generación de variables aleatorias

Universidad	Universidad de los Llanos, Escuela de Ingeniería
Curso	Simulación Computacional
Docente	Angel Alfonso Cruz Roa
Estudiante	Luciana Alejandra Belandria Agudelo
Código	160005005

1. Objetivos

- Introducir a los métodos para la generación y validación de números pseudoaleatorios.
- Generar secuencias de números pseudoaleatorios usando el método de MidSquare.
- Generar secuencias de números pseudoaleatorios usando el método del congruencial mixto.
- Generar secuencias de números pseudoaleatorios usando otros métodos.
- Calcular el tamaño de ciclo o periodo de una secuencia de números pseudoaleatorios.
- Evaluar si una secuencia de números pseudoaleatorios cumple con los criterios de uniformidad y aleatoriedad.
- Generar secuencias de valores de una variable aleatoria de una distribución de probabilidad discreta usando el método de la transformada inversa.

2. Consulta previa

Una **variable aleatoria** es simplemente una regla que le pone un número a cada resultado posible de un experimento cuyo desenlace no se conoce de antemano. Si uno lanza dos dados, el experimento tiene 36 desenlaces posibles y la suma de las caras es una variable aleatoria, porque a cada desenlace le asigna un número entre 2 y 12. Lo importante es que uno no sabe qué valor va a salir, pero sí sabe qué valores puede tomar y con qué frecuencia aparece cada uno.

Esa segunda parte, la de las frecuencias, es justamente lo que describe una **distribución de probabilidad**. Es la función que reparte la probabilidad entre los valores que la variable puede tomar, con la condición de que todas las probabilidades sean no negativas y de que sumen (o integren) exactamente uno. Es, en el fondo, el mapa completo del comportamiento de la variable.

La diferencia entre una **distribución discreta** y una **continua** está en el tipo de valores que la variable puede tomar. Cuando los valores son contables, como el número de clientes que llegan en una hora, la distribución es discreta y se describe con una función de masa de probabilidad $p(x) = P\{X = x\}$, donde cada valor individual tiene una probabilidad propia mayor que cero. Cuando la variable puede tomar cualquier valor dentro de un intervalo, como el tiempo que dura una llamada, la distribución es continua y se describe con una función de densidad $f(x)$; ahí la

probabilidad de que la variable valga exactamente un número puntual es cero, y lo único que tiene sentido preguntar es la probabilidad de caer dentro de un rango, que se calcula integrando la densidad.

La distribución que más se usa en simulación es la **uniforme continua en el intervalo** $(0, 1)$, que se escribe $U(0, 1)$. Su densidad vale uno dentro del intervalo y cero fuera de él,

$$f(x) = \begin{cases} 1, & 0 < x < 1 \\ 0, & \text{en otro caso} \end{cases} \quad F(x) = \begin{cases} 0, & x \leq 0 \\ x, & 0 < x < 1 \\ 1, & x \geq 1 \end{cases}$$

con media $1/2$ y varianza $1/12$. La idea detrás es que ningún pedazo del intervalo está favorecido frente a otro del mismo ancho. Esta distribución es la materia prima de todo el laboratorio, porque a partir de números $U(0, 1)$ se puede construir prácticamente cualquier otra variable aleatoria.

La **distribución de Poisson** sirve para contar cuántas veces ocurre algo en un intervalo fijo de tiempo o espacio, cuando esos eventos suceden de manera independiente y a una tasa promedio constante. Su función de masa es

$$P\{X = i\} = e^{-\lambda} \frac{\lambda^i}{i!}, \quad i = 0, 1, 2, \dots$$

donde λ es al mismo tiempo la media y la varianza. Un detalle práctico útil es que se puede generar de forma recursiva, ya que $p_{i+1} = \frac{\lambda}{i+1} p_i$ partiendo de $p_0 = e^{-\lambda}$, lo cual evita calcular factoriales.

La **distribución binomial** cuenta cuántos éxitos se obtienen en n intentos independientes cuando cada intento tiene la misma probabilidad de éxito p . Su función de masa es

$$P\{X = i\} = \binom{n}{i} p^i (1-p)^{n-i}, \quad i = 0, 1, \dots, n$$

con media np y varianza $np(1-p)$. También tiene una recursión cómoda, $p_{i+1} = \frac{p}{1-p} \cdot \frac{n-i}{i+1} p_i$ arrancando en $p_0 = (1-p)^n$, y esa es exactamente la que se implementó en la sección 5.4.

Cuando se dice que una secuencia de variables es **independiente e idénticamente distribuida**, o i.i.d., se están afirmando dos cosas distintas a la vez. Lo de idénticamente distribuida quiere decir que todas salen de la misma distribución, con los mismos parámetros; lo de independiente quiere decir que saber el valor de una no da ninguna pista sobre las demás. Esta propiedad es la que uno le exige a un buen generador, y es también la que justifica que el promedio de una muestra grande se acerque al valor esperado, que es el principio sobre el que descansa Monte Carlo.

Finalmente, una **prueba o test estadístico** es un procedimiento para decidir, con evidencia muestral, si una afirmación sobre una población es creíble o no. Uno plantea una hipótesis nula H_0 , que en este laboratorio suele ser «los números son uniformes» o «los números son aleatorios», calcula un estadístico a partir de los datos y compara ese valor con lo que uno esperaría si H_0 fuera cierta. Si el estadístico cae en una zona muy improbable, se rechaza H_0 ; si no, se acepta provisionalmente. Hay que tener claro que aceptar no es demostrar, solo significa que los datos no dieron motivos para dudar, y que siempre existe la posibilidad de rechazar algo cierto por pura mala suerte, lo cual se controla con el nivel de significancia α , que aquí se fijó en 0.05.

3. Generación de números (pseudo)aleatorios

3.1. Cuadrados medios y generador congruencial mixto

El primer método que se implementó fue el de **cuadrados medios** o MidSquare, propuesto por Von Neumann, que es el más intuitivo de todos. Se parte de una semilla de d dígitos, se eleva al cuadrado, se rellena el resultado con ceros a la izquierda hasta completar $2d$ dígitos, se toman los d dígitos del centro y esos forman el siguiente número. Dividiendo entre 10^d queda un valor en el intervalo $(0, 1)$.

```
def midSquare(xant, d):
    cuadrado = str(xant**2).zfill(2*d)
    ini = (2*d - d)//2
    xsig = int(cuadrado[ini:ini+d])
    usig = xsig / (10**d)
    return [xsig, usig]
```

Listing 1: Método de cuadrados medios

Se generaron secuencias de $n = 100$ números con las cinco semillas pedidas por la guía, que fueron 1931, 675248, 4567, 12122 y 7179345. Ya desde aquí aparece el problema grande de este método, y es que se muere solo. Con la semilla 1931 la secuencia cae en el valor cero después de apenas siete pasos y de ahí no vuelve a salir, porque el cuadrado de cero es cero.

El segundo método fue el **generador congruencial mixto**, también llamado generador lineal congruencial o GLC, que es el que realmente se usa en la práctica. Su recurrencia es

$$X_{n+1} = (aX_n + c) \text{ mód } m, \quad u_i = \frac{X_i}{m}$$

y se implementó con el código del notebook *Números aleatorios* del curso.

```
def genran(a, c, m, xant):
    xsig = (a * xant + c) % m
    usig = xsig / m
    return [xsig, usig]

def genranN(a, c, m, x0, N):
    xant = x0
    I, X, U = [], [], []
    for t in range(1, N + 1):
        [xi, ui] = genran(a, c, m, xant)
        I.append(t); X.append(xi); U.append(ui)
        xant = xi
    return [I, X, U]
```

Listing 2: Generador congruencial mixto

Sobre este código hay que hacer una aclaración de implementación que vale la pena mencionar. En el notebook original los parámetros están escritos como flotantes; para los juegos con $m = 2^{31} - 1$ y $m = 2^{48}$ el producto aX_n supera los 53 bits de mantisa de un `float64` y la recurrencia empieza a dar resultados equivocados por pérdida de precisión. Por eso aquí se trabajó con enteros de Python, que tienen precisión arbitraria y dejan la recurrencia exacta.

Con ese generador se produjeron las cinco secuencias de $n = 100$ números que pide la guía, usando los parámetros de la tabla 1.

Tabla 1: Juegos de parámetros del generador congruencial mixto

	X_0	a	c	m
a	127	115	0	128
b	43	25	5	48
c	13	16807	0	$2^{31} - 1$
d	29022024	25214903917	11	2^{48}
e	3	5	7	200

3.2. Otros generadores

La guía pedía implementar al menos dos generadores adicionales de los que aparecen en el artículo de Mancilla. Se implementaron los cuatro, con parámetros escogidos por el grupo.

El **Shift-Register** trabaja directamente con bits. Cada bit nuevo sale de combinar con un XOR dos bits anteriores del registro, siguiendo la regla $x_n = (x_{n-k} + x_{n-q}) \bmod 2$; aquí se usó $k = 31$ y $q = 3$, que corresponden al polinomio primitivo $x^{31} + x^3 + 1$, y se tomaron 31 bits sucesivos para armar cada entero. El **Lagged-Fibonacci** funciona con una tabla de valores anteriores y calcula $X_n = (X_{n-j} + X_{n-k}) \bmod M$ con retardos $j = 5$ y $k = 17$ y módulo $M = 2^{31}$, llenando la tabla inicial con un GLC. El de **congruencia inversa** cambia el término lineal por el inverso multiplicativo módulo m , es decir $X_n = a\bar{X}_{n-1} + b \pmod{m}$ con $m = 2^{31} - 1$ primo, y el inverso se calcula con el pequeño teorema de Fermat. Por último, el **congruencial lineal combinado** corre dos GLC en paralelo con los parámetros clásicos de L'Ecuyer y suma sus salidas módulo el mayor de los dos módulos, con lo que el período se dispara al orden de $M_1 M_2 \approx 4.6 \times 10^{18}$.

3.3. Tamaño de ciclo y pruebas estadísticas

Con las catorce secuencias en la mano se pasó a evaluarlas. Lo primero fue el **tamaño de ciclo**. Como todos estos generadores son funciones deterministas sobre un conjunto finito de estados, tarde o temprano repiten un estado y a partir de ahí la secuencia se vuelve cíclica. Guardando en un diccionario la primera aparición de cada estado se obtienen dos números, el transitorio μ , que son los términos que ocurren antes de entrar al ciclo, y el período λ , que es la longitud del ciclo. Cuando $\mu > 0$ se dice que la sucesión se degenera, porque la semilla nunca vuelve a aparecer. Para los generadores con módulos enormes se fijó un tope de búsqueda de 2×10^5 iteraciones y se reporta que el período lo supera.

Después vinieron las pruebas. Para uniformidad se aplicaron tres. El contraste χ^2 divide el intervalo $(0, 1)$ en $k = 10$ clases iguales y compara las frecuencias observadas con las $e_i = n/k = 10$ que se esperarían si los números estuvieran bien repartidos,

$$\chi^2 = \sum_{i=1}^k \frac{(f_i - e_i)^2}{e_i} \sim \chi_{k-1}^2$$

El contraste de Kolmogorov-Smirnov compara la función de distribución empírica contra la teórica y se queda con la mayor diferencia,

$$D_n = \max_{1 \leq i \leq n} \left\{ \max \left[\left| \frac{i}{n} - x_{(i)} \right|, \left| x_{(i)} - \frac{i-1}{n} \right| \right] \right\}$$

Y la prueba de pares consecutivos no solapados agrupa los valores discretizados de dos en dos y verifica que todas las parejas (i, j) aparezcan con la misma frecuencia esperada $E_{ij} = n/(2k^2)$, lo

cual detecta estructuras que la uniformidad simple no ve. Aquí se usó $k = 3$ en lugar de $k = 10$, porque con $n = 100$ y diez clases quedaría $E_{ij} = 0.5$ y la aproximación χ^2 exige frecuencias esperadas mayores que 5; con tres clases queda $E_{ij} = 5.56$, que ya es aceptable.

Para aleatoriedad se aplicó el **test de rachas**. Se construye una secuencia de ceros y unos según si cada número es menor o mayor que el anterior, se cuentan los bloques consecutivos del mismo símbolo y se compara ese conteo con lo que se esperaría por azar,

$$Z = \frac{R - \frac{2n-1}{3}}{\sqrt{\frac{16n-29}{90}}} \sim \mathcal{N}(0, 1)$$

Sobre el código base del notebook de rachas se hizo una corrección, y es que ese código obtiene las longitudes a partir de las posiciones donde hay cambio de signo, con lo cual deja por fuera la última racha; aquí se contó R como el número de cambios más uno.

Los resultados de todo lo anterior están reunidos en la tabla 2, donde OK significa que se acepta la hipótesis y REC que se rechaza al nivel $\alpha = 0.05$. Los valores críticos usados fueron $\chi_{9;0.05}^2 = 16.919$, $D_{crit}(100) = 0.1340$, $\chi_{8;0.05}^2 = 15.507$ y $Z_{\alpha/2} = 1.960$.

Tabla 2: Tamaño de ciclo y pruebas para las catorce secuencias de $n = 100$ números

Generador	Dist.	μ	λ	χ^2	D	T pares	Z rachas
MidSquare 1931	7	7	1	860.60 REC	0.9740 REC	382.36 REC	-15.398 REC
MidSquare 675248	100	210	1	12.40 OK	0.0820 OK	10.48 OK	0.878 OK
MidSquare 4567	22	19	4	109.40 REC	0.1700 REC	126.04 REC	-4.388 REC
MidSquare 12122	88	88	1	52.20 REC	0.2166 REC	19.12 REC	-2.952 REC
MidSquare 7179345	100	848	20	15.00 OK	0.1043 OK	3.64 OK	-0.080 OK
GLC (a)	32	0	32	6.80 OK	0.0441 OK	6.88 OK	0.638 OK
GLC (b)	48	0	48	1.20 OK	0.0358 OK	36.04 REC	1.117 OK
GLC (c)	100	-	$> 2 \times 10^5$	12.00 OK	0.1452 REC	11.56 OK	-0.558 OK
GLC (d)	100	-	$> 2 \times 10^5$	4.80 OK	0.0831 OK	5.80 OK	-0.798 OK
GLC (e)	9	2	8	22.80 REC	0.0950 OK	58.36 REC	1.835 OK
Shift-Register	100	-	$> 2 \times 10^4$	5.80 OK	0.0526 OK	4.00 OK	0.399 OK
Lagged-Fibonacci	100	-	$> 2 \times 10^5$	13.20 OK	0.1041 OK	11.20 OK	-0.558 OK
Congruencia inversa	100	-	$> 2 \times 10^5$	6.60 OK	0.0780 OK	4.72 OK	0.160 OK
GLC combinado	100	-	$> 2 \times 10^5$	3.20 OK	0.0927 OK	4.00 OK	0.399 OK

3.4. Cuáles son los peores generadores y por qué

El peor de todos, sin discusión, es el **MidSquare con semilla 1931**. La secuencia se degenera en siete pasos y cae en el punto fijo cero, de modo que su período es $\lambda = 1$ y de los cien números generados solo hay siete distintos, el resto son ceros. Rechaza las cuatro pruebas por márgenes enormes. Detrás vienen las semillas 4567, que tiene transitorio 19 y período 4 con apenas 22 valores distintos, y 12122, que también colapsa en un punto fijo después de 88 pasos. Las tres rechazan uniformidad y aleatoriedad.

También queda muy mal el **GLC con** $X_0 = 3$, $a = 5$, $c = 7$, $m = 200$, que se degenera con transitorio 2 y período 8, es decir que produce solo nueve valores distintos. La razón se explica con las condiciones de período máximo del artículo de Mancilla, ya que $m = 200 = 2^3 \cdot 5^2$ y, aunque $c = 7$ sí es primo relativo con m , resulta que $a - 1 = 4$ no es múltiplo de 5, que es un factor primo de m . Rechaza χ^2 y pares consecutivos.

Un caso más sutil son los **GLC (a) y (b)**. Ninguno se degenera, y el (b) incluso alcanza el período máximo posible para $m = 48$, pero sus períodos de 32 y 48 son tan cortos que en cien números la secuencia completa se repite dos o tres veces. Aunque pasan χ^2 , Kolmogorov-Smirnov y rachas, precisamente porque recorrer el ciclo completo se ve muy uniforme, el (b) rechaza la prueba de pares consecutivos, que sí alcanza a detectar la repetición. La conclusión que deja este par es que un generador con período corto no sirve para simulación aunque pase las pruebas de uniformidad.

Vale la pena señalar un caso engañoso. El **MidSquare con semillas 675248 y 7179345** pasa las cuatro pruebas, pero no por eso es bueno. Sus transitorios son de 210 y 848 pasos respectivamente, es decir más largos que los cien números pedidos, así que la degeneración existe pero todavía no alcanza a verse en la muestra. Sus períodos reales son 1 y 20. Esto muestra justamente por qué el tamaño de ciclo tiene que analizarse aparte de las pruebas estadísticas y no confiar solo en ellas.

3.5. Cuáles son los mejores generadores y por qué

El mejor del conjunto es el **GLC (d)**, con $X_0 = 29022024$, $a = 25214903917$, $c = 11$ y $m = 2^{48}$, que son los parámetros de la función **drand48** de C. No se degenera, su período supera con creces el tope de búsqueda y teóricamente vale $2^{48} \approx 2.8 \times 10^{14}$, porque cumple las condiciones de período máximo, esto es, c impar y primo relativo con m , y $a - 1$ múltiplo de 4. Además pasa las cuatro pruebas con holgura.

Muy cerca quedan los **cuatro generadores adicionales** implementados, que también pasan todas las pruebas y tienen períodos gigantescos. El combinado y el de congruencia inversa son los que mejor se comportan cuando se grafican los pares (u_i, u_{i+1}) , donde no se alcanza a ver estructura reticular.

Mención aparte merece el **GLC (c)**, el clásico MINSTD de Lehmer con $a = 16807$ y $m = 2^{31} - 1$. Tiene período enorme y pasa χ^2 , pares consecutivos y rachas, pero rechaza Kolmogorov-Smirnov por muy poco, con $D = 0.1452$ frente a un crítico de 0.1340. Con apenas cien números y $\alpha = 0.05$, una prueba puede rechazar por azar aproximadamente una de cada veinte veces, así que lo correcto antes de descartarlo es repetir el contraste con otras semillas o con un n más grande.

En resumen, el criterio que más discrimina en este laboratorio no son las pruebas estadísticas sino el tamaño de ciclo. Todos los generadores rechazados o dudosos tienen períodos ridículamente cortos o se degeneran, mientras que los que tienen módulo grande y parámetros que cumplen las condiciones de período máximo pasan todo sin problema.

4. Uso de números aleatorios para evaluar integrales

Habiendo comprobado cuáles generadores sirven, el siguiente paso fue ponerlos a trabajar. La idea de Monte Carlo es reescribir una integral como el valor esperado de una función de una variable uniforme y luego aproximar ese valor esperado por un promedio muestral. Si $U \sim U(0, 1)$, entonces

$$\theta = \int_0^1 g(x) dx = E[g(U)] \approx \frac{1}{N} \sum_{i=1}^N g(u_i)$$

y por la ley de los grandes números el promedio converge al valor real a medida que N crece.

Cuando los límites no son cero y uno hay que acomodar la integral. Para un intervalo finito $[a, b]$ se hace el cambio $x = a + (b-a)u$, con lo cual $dx = (b-a) du$ y la integral queda $(b-a)E[g(a + (b-a)U)]$.

Para un límite infinito se usa el cambio $x = \frac{1}{u} - 1$, que manda el intervalo $(0, 1)$ sobre $(0, \infty)$ y tiene $dx = du/u^2$, de modo que

$$\int_0^\infty g(x) dx = \int_0^1 \frac{g(\frac{1}{u} - 1)}{u^2} du$$

Y cuando el dominio es toda la recta se aprovecha la simetría de la función para escribirla como el doble de la integral en $(0, \infty)$.

Todo eso se empaquetó en una sola función que recibe la función a integrar, el tamaño de muestra, los límites y el tipo de transformación, y que se alimenta con el generador congruencial mixto en lugar de con `numpy.random`, para dejar claro que los números vienen de la implementación propia.

```
def montecarlo(g, N, a=0.0, b=1.0, tipo='ab', x0=12345):
    _, _, U = genranN(A_MC, C_MC, M_MC, x0, N)
    U = np.array(U)
    if tipo == '01':
        muestras = g(U)
    elif tipo == 'ab':
        muestras = (b - a) * g(a + (b - a) * U)
    elif tipo == 'inf':
        muestras = g(1.0/U - 1.0) / (U**2)
    elif tipo == 'inf_sim':
        muestras = 2.0 * g(1.0/U - 1.0) / (U**2)
    est = np.mean(muestras)
    err = np.std(muestras, ddof=1) / np.sqrt(N)
    return est, err
```

Listing 3: Estimador general de Monte Carlo

Antes de resolver los ejercicios se corrió el código base de la sección 3.2 sobre $\int_0^2 e^{-x^2} dx$, cuyo valor real es 0.882081, y se obtuvieron estimaciones de 0.5460 con diez muestras, 0.8903 con diez mil y 0.8812 con un millón, lo cual ya muestra el comportamiento típico del método. Con pocas muestras el resultado es bastante malo, y la mejora es lenta pero sostenida.

Los ejercicios 3 al 9 se resolvieron con la misma maquinaria. Los ejercicios 3 y 4 son directos sobre $(0, 1)$. El 5 necesita el cambio de intervalo con $a = -2$ y $b = 2$. El 6 y el 7 requieren la transformación de límite infinito, y el 7 además la simetría de e^{-x^2} . El 8 es una integral doble sobre el cuadrado unitario, así que se generaron dos secuencias independientes con semillas distintas y se promedió $e^{(u_i+v_i)^2}$ directamente, ya que el área de la región es uno. El 9 fue el más elaborado, porque hubo que aplicar primero la sugerencia de la guía, es decir usar la función indicadora $I_y(x)$ que vale uno si $y < x$ y cero en caso contrario, para poder extender el límite interno hasta infinito, y después aplicar el cambio de variable en las dos direcciones. Los resultados aparecen en la tabla 3.

Tabla 3: Estimaciones de Monte Carlo para los ejercicios 3 al 9

Integral	Valor exacto	$N = 10^2$	$N = 10^3$	$N = 10^4$	$N = 10^5$	$N = 10^6$
3. $\int_0^1 e^{e^x} dx$	6.316564	6.624227	6.369560	6.293961	6.312443	6.316263
4. $\int_0^1 (1 - x^2)^{3/2} dx$	0.589049	0.546212	0.581614	0.590294	0.589258	0.589085
5. $\int_{-2}^2 e^{x+x^2} dx$	93.162753	88.901472	94.847830	90.374996	92.790622	93.043249
6. $\int_0^\infty x(1+x^2)^{-2} dx$	0.500000	0.527945	0.509402	0.500663	0.499529	0.499432
7. $\int_{-\infty}^\infty e^{-x^2} dx$	1.772454	2.015744	1.821253	1.780836	1.774152	1.771025
8. $\int_0^1 \int_0^1 e^{(x+y)^2} dy dx$	4.899159	4.071028	4.668887	4.832847	4.909325	4.908658
9. $\int_0^\infty \int_0^x e^{-(x+y)} dy dx$	0.500000	0.507368	0.494095	0.489846	0.500733	0.498913

Lo que salta a la vista es que en todos los casos la estimación se acerca al valor real conforme crece

N , pero no de manera perfectamente monótona, y eso es lo esperable porque el método es aleatorio y cada corrida es una muestra distinta. También se nota que no todas las integrales convergen igual de rápido. El ejercicio 5 es el más difícil de los siete, porque e^{x+x^2} crece muchísimo cerca del extremo derecho del intervalo y eso dispara la varianza del estimador; con cien muestras el error estándar es de casi 22 unidades, y aun con un millón de muestras la estimación todavía se aleja una décima del valor real. En el otro extremo están los ejercicios 4 y 6, donde la función es suave y acotada y ya con diez mil muestras el error es de milésimas.

El error estándar que devuelve la función confirma la teoría, ya que se reduce aproximadamente a la mitad cada vez que N se multiplica por cuatro, tal como predice el factor $1/\sqrt{N}$. Eso es lo bueno y lo malo de Monte Carlo al mismo tiempo, porque garantiza convergencia sin importar la dimensión del problema, pero exige cien veces más muestras para ganar un dígito de precisión.

5. Generación de variables aleatorias discretas

Con los números uniformes ya validados y aplicados a integrales, el siguiente paso natural es usarlos para fabricar valores de otras distribuciones. El método de la transformada inversa para el caso discreto es directo, ya que se genera $U \sim U(0,1)$ y se recorre la distribución acumulada hasta encontrar el primer valor que la supera, es decir se toma

$$X = x_j \quad \text{si} \quad \sum_{i=1}^{j-1} p_i \leq U < \sum_{i=1}^j p_i$$

La idea es simple, ya que a cada valor posible se le asigna un pedazo del intervalo $(0,1)$ proporcional a su probabilidad, y como U cae uniformemente en ese intervalo, la frecuencia con que cae en cada pedazo reproduce la distribución deseada.

```
def genvardiscret(U, X, P):
    V = []
    for t in range(0, len(U)):
        for t2 in range(0, len(X)):
            if U[t] < P[t2]:
                V.append(X[t2])
                break
    return V
```

Listing 4: Transformada inversa para variables discretas

Toda esta sección se alimentó con el generador congruencial mixto usando los parámetros que exige la guía, o sea $X_0 = 2391$, $a = 25214903917$, $c = 11$ y $m = 2^{48}$, que corresponden justamente al mejor generador identificado en la sección anterior. Los primeros diez números uniformes producidos fueron 0.2142, 0.7450, 0.6610, 0.7595, 0.7835, 0.3924, 0.6607, 0.8984, 0.3532 y 0.7253.

Con esa secuencia se generaron los primeros cien valores de las dos distribuciones pedidas en los literales a y b. En el primer caso las probabilidades son 0.20, 0.15, 0.25 y 0.40, que acumuladas dan los cortes 0.20, 0.35, 0.60 y 1.00; en el segundo son 0.30, 0.20, 0.35 y 0.15, con cortes 0.30, 0.50, 0.85 y 1.00. La comparación entre lo que salió y lo que se esperaba está en la tabla 4.

Tabla 4: Proporciones empíricas frente a las teóricas con $n = 100$

	$X = 1$	$X = 2$	$X = 3$	$X = 4$
Literal a, teórica	0.20	0.15	0.25	0.40
Literal a, empírica	0.21	0.14	0.27	0.38
Literal b, teórica	0.30	0.20	0.35	0.15
Literal b, empírica	0.32	0.22	0.29	0.17

El ajuste es bueno en ambos casos, con desviaciones que no pasan de seis centésimas. Conviene recordar que con cien observaciones la desviación estándar de una proporción ronda las cuatro o cinco centésimas, así que estas diferencias están dentro de lo normal y no indican ningún problema del generador.

El literal c pedía algo parecido pero variando el tamaño de la muestra, con una variable que toma el valor 1 con probabilidad $1/3$ y el valor 2 con probabilidad $2/3$, para ver cómo se comporta la proporción de unos. Los resultados fueron 0.3400 con cien valores, 0.3280 con mil y 0.3316 con diez mil, frente al valor teórico 0.3333. Aquí se ve con mucha claridad la ley de los grandes números, porque el error va bajando de seis milésimas a cinco y luego a menos de dos a medida que la muestra crece cien veces.

El literal d cambió de método. Se pedía generar una variable que toma los valores del 1 al 10 con probabilidades 0.06 para los cinco primeros y 0.15, 0.13, 0.14, 0.15 y 0.13 para los cinco últimos, usando **composición** en lugar de transformada inversa directa. La gracia del método de composición es que descompone la distribución como una mezcla de distribuciones más sencillas. Aquí se escribió

$$p(x) = 0.30 p_1(x) + 0.70 p_2(x)$$

donde la primera componente reparte 0.30 de probabilidad de manera uniforme entre los valores del 1 al 5, ya que $0.06/0.30 = 0.2$ para cada uno, y la segunda reparte el 0.70 restante entre los valores del 6 al 10 según sus probabilidades renormalizadas. El algoritmo entonces usa un primer número uniforme para escoger la componente y un segundo para escoger el valor dentro de ella.

```
def gen_composicion(U1, U2):
    w1 = 0.30
    cond1 = np.cumsum([0.2, 0.2, 0.2, 0.2, 0.2])
    cond2 = np.cumsum(np.array([0.15, 0.13, 0.14, 0.15, 0.13]) / 0.70)
    V = []
    for u1, u2 in zip(U1, U2):
        if u1 < w1:
            for j in range(5):
                if u2 < cond1[j]:
                    V.append(1 + j); break
        else:
            for j in range(5):
                if u2 < cond2[j]:
                    V.append(6 + j); break
    return V
```

Listing 5: Método de composición

Las proporciones obtenidas en cien valores fueron 0.08, 0.06, 0.04, 0.09, 0.05, 0.14, 0.08, 0.13, 0.22 y 0.11 frente a las teóricas. El grueso de la distribución quedó bien reproducido, ya que la mezcla asignó 32 valores al primer bloque contra los 30 esperados, pero dentro de cada bloque hay desviaciones

notorias, sobre todo en el valor 9, que salió con frecuencia 0.22 cuando debía rondar 0.15. Con cien muestras la desviación estándar de esa proporción es de unas tres centésimas y media, así que la diferencia equivale a dos desviaciones, lo cual es alto pero todavía dentro de lo que puede pasar por azar. Con una muestra más grande esas irregularidades desaparecerían.

6. Simulación ad hoc con variables aleatorias discretas

La última parte junta todo lo anterior. Se retomó la simulación de la fila de un solo servidor del Laboratorio 1, con la misma tabla de nueve columnas del capítulo 1 de Banks, pero cambiando el origen de los tiempos. Antes los tiempos entre llegadas salían de una uniforme discreta entre 1 y 10 y los tiempos de servicio de una uniforme discreta entre 1 y 6, es decir de dados. Ahora los tiempos entre llegadas se generan con una variable de Poisson con $\lambda = 10$ y los de servicio con una binomial de $n = 10$ y $p = 0.40$, ambas implementadas con las recursiones mencionadas en la consulta previa.

```
def genpoisson(U, L):
    i = 0
    px = np.exp(-L)
    Fx = px
    while True:
        if U < Fx:
            return i
        px = (L*px)/(i+1)
        Fx = Fx + px
        i = i + 1

def genbinomial(U, n, p):
    c = p/(1-p)
    i = 0
    px = np.power(1-p, n)
    Fx = px
    while True:
        if U < Fx:
            return i
        px = (c*(n-i)/(i+1))*px
        Fx = Fx + px
        i = i + 1
```

Listing 6: Generadores de Poisson y binomial por transformada inversa

La lógica de la fila es la misma de siempre. El cliente 1 llega en el tiempo cero y por eso no tiene tiempo entre llegadas. Para los demás, la hora de llegada es la del cliente anterior más su tiempo entre llegadas, el servicio empieza en el máximo entre la hora de llegada y la hora en que se desocupó el servidor, y termina sumando el tiempo de servicio. El tiempo en el sistema es la diferencia entre el fin del servicio y la llegada, el tiempo ocioso es lo que el servidor esperó sin nadie y el tiempo en cola es lo que el cliente esperó a que se desocupara. Se simularon 20 clientes y la tabla 5 muestra el resultado.

Tabla 5: Simulación de la fila con 20 clientes

Cliente	TEL	Llegada	TS	Inicio serv.	Fin serv.	En sistema	Ocio	En cola
1	—	0	4	0	4	4	0	0
2	12	12	3	12	15	3	8	0
3	11	23	4	23	27	4	8	0
4	12	35	3	35	38	3	8	0
5	12	47	5	47	52	5	9	0
6	9	56	5	56	61	5	4	0
7	11	67	3	67	70	3	6	0
8	14	81	2	81	83	2	11	0
9	9	90	3	90	93	3	7	0
10	12	102	3	102	105	3	9	0
11	8	110	6	110	116	6	5	0
12	4	114	2	116	118	4	0	2
13	12	126	2	126	128	2	8	0
14	12	138	4	138	142	4	10	0
15	6	144	6	144	150	6	2	0
16	7	151	7	151	158	7	1	0
17	7	158	4	158	162	4	0	0
18	6	164	4	164	168	4	2	0
19	7	171	3	171	174	3	3	0
20	9	180	1	180	181	1	6	0
Sumas						76	107	2

Sobre esa tabla se calcularon las cinco medidas de desempeño del capítulo 1 de Banks, con las mismas fórmulas del Laboratorio 1. El tiempo promedio en el sistema fue de 3.800 unidades, el porcentaje de tiempo ocioso del servidor fue de 59.12 %, el tiempo promedio de espera por cliente fue de 0.100, la fracción de clientes que tuvieron que esperar fue de 0.050 y el tiempo promedio de espera de los que sí esperaron fue de 2.000. Como control, la media simulada de los tiempos entre llegadas dio 9.474 frente al valor teórico $\lambda = 10$, y la media simulada de los tiempos de servicio dio 3.700 frente al teórico $np = 4.0$, lo cual confirma que los generadores están produciendo lo que deben dentro de la variabilidad esperable para 20 observaciones.

6.1. Qué se puede decir de los resultados

Lo primero que se nota es que el servidor pasa desocupado más de la mitad del tiempo, casi un 60 %, y que el sistema prácticamente no se congestiona. Solo un cliente de veinte tuvo que esperar, y esperó dos unidades, lo cual deja un tiempo de espera promedio por cliente de apenas una décima. Como consecuencia, el tiempo promedio en el sistema, que fue de 3.800, está definido casi por completo por el tiempo de servicio promedio de 3.700, porque la gran mayoría de los clientes encuentra el servidor libre apenas llega.

Esto no es casualidad sino consecuencia directa de los parámetros escogidos. La utilización teórica del sistema es el cociente entre el tiempo de servicio esperado y el tiempo entre llegadas esperado, o sea $np/\lambda = 4/10 = 0.40$, y una utilización de 0.40 describe un sistema muy holgado, donde llega un cliente cada diez unidades de tiempo y atenderlo cuesta apenas cuatro.

6.2. Comparación con el Laboratorio 1

En cuanto a las similitudes, ambas simulaciones conservan la misma estructura de nueve columnas de la tabla 1.1 de Banks para registrar el reloj de llegada, el inicio y el fin del servicio, y los tiempos

de ocio y espera. También se evalúan exactamente las mismas cinco medidas de desempeño, y en los dos casos los valores generados tanto para las llegadas como para el servicio se mantienen como números enteros, así que la mecánica del recorrido cliente por cliente no cambia en absoluto.

Las diferencias están en las distribuciones y en lo que ellas arrastran. El Laboratorio 1 usaba únicamente uniformes discretas, mientras que esta simulación usa Poisson y binomial, y ese cambio altera por completo la carga del sistema. La configuración del Laboratorio 1 generaba llegadas mucho más frecuentes frente al tiempo de servicio, con una utilización cercana a 0.64, mientras que aquí las llegadas se separan y la carga baja a 0.40. Como consecuencia directa de esa menor carga, el tiempo ocioso se dispara al 59.12 % frente al 33.4 % promedio que daban las corridas de veinte clientes del primer laboratorio, y la congestión cae en picada, con una fracción de clientes que esperan de apenas 5 % contra el 31.5 % promedio reportado antes.

Hay una diferencia conceptual que también vale la pena anotar. Usar una variable de Poisson para representar el tiempo *entre* llegadas no es lo más ortodoxo, ya que Poisson naturalmente cuenta eventos por unidad de tiempo y lo que suele modelar el tiempo entre llegadas es la exponencial. La guía lo pide así, y funciona porque los valores que entrega son enteros positivos con media λ , pero conviene tener presente que si se quisiera modelar un proceso de llegadas de Poisson en sentido estricto habría que generar tiempos exponenciales, cosa que corresponde al método de la transformada inversa en el caso continuo y que queda fuera del alcance de este laboratorio.

7. Conclusiones

El recorrido completo deja varias ideas claras. La primera es que la calidad de un generador de números pseudoaleatorios no se puede juzgar solo con pruebas estadísticas, porque el tamaño de ciclo es un criterio independiente y a veces más severo. El caso de MidSquare con las semillas 675248 y 7179345, que pasa las cuatro pruebas pero está condenado a degenerarse, lo ilustra perfectamente, y lo mismo pasa al revés con el MINSTD de Lehmer, que es un generador excelente y sin embargo falló una prueba por azar.

La segunda es que en el generador congruencial mixto todo depende de escoger bien los cuatro parámetros. Con módulo pequeño o con parámetros que no cumplen las condiciones de período máximo el generador es inservible, mientras que con módulo grande y parámetros correctos el comportamiento es impecable. No es cuestión de suerte sino de teoría de números.

La tercera es que una vez se tiene una buena fuente de números uniformes, todo lo demás se construye encima con relativamente poco esfuerzo. Monte Carlo para integrales, transformada inversa para variables discretas, composición para mezclas y los generadores de Poisson y binomial son todos aplicaciones del mismo insumo, y de ahí la insistencia del laboratorio en validar primero la base antes de construir sobre ella.

La última es que la aleatoriedad tiene un costo que hay que aceptar. Monte Carlo converge siempre, pero lentamente, con un error que baja como $1/\sqrt{N}$, y las simulaciones cortas de veinte clientes producen medidas de desempeño con mucha variabilidad. Para sacar conclusiones firmes sobre un sistema de colas haría falta correr muchas réplicas y reportar intervalos de confianza, no una sola corrida.

Bibliografía

1. Ross, Sheldon. *Simulación*, 2.^a edición. Pearson Press, 1999. Capítulos 3 y 4.

2. Mancilla Herrera, Alfonso Manuel. «Números aleatorios. Historia, teoría y aplicaciones». *Ingeniería y Desarrollo*, núm. 8, diciembre de 2000, pp. 49-69. Universidad del Norte, Barranquilla, Colombia.
3. Banks, Jerry; Carson, John; Nelson, Barry; Nicol, David. *Discrete-Event System Simulation*. Prentice Hall. Capítulo 1.
4. Ríos Insua, David; Ríos Insua, Sixto; Martín, Jacinto. *Simulación. Métodos y aplicaciones*. Alfaomega, 2000.
5. Cruz Roa, Angel Alfonso. Notebooks del curso Simulación Computacional. Universidad de los Llanos, 2025.